

LLM Fundamentals Manual (Simple Hinglish)

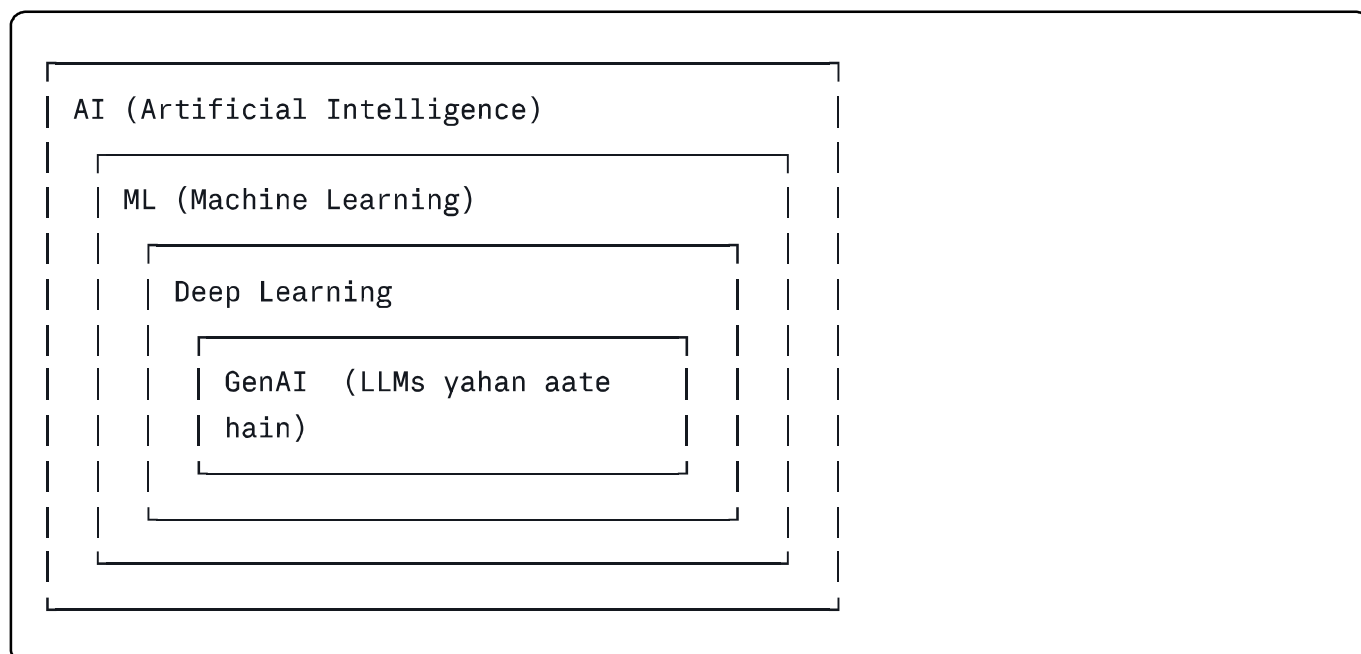
Ye manual un logon ke liye hai jo LLM (Large Language Model) ko bilkul basic se samajhna chahte hain. Koi heavy maths nahi, sirf clear concepts aur examples.

Table of Contents

1. AI vs ML vs Deep Learning vs GenAI
 2. LLM kya hota hai?
 3. LLM kaise kaam karta hai? (Next Token Prediction)
 4. Tokens aur Tokenization
 5. Embeddings
 6. Transformer aur Attention
 7. LLM ki training kaise hoti hai?
 8. Parameters aur Model Size
 9. Context Window
 10. Inference (Model se Output Kaise Milta Hai)
 11. System / User / Assistant Messages
 12. Generation Settings
 13. Prompting Basics
 14. Hallucination
 15. RAG (Retrieval-Augmented Generation)
 16. Prompting vs RAG vs Fine-tuning
 17. Tool Use aur AI Agents
 18. LLM ki Limitations
 19. Safety aur Best Practices
 20. Glossary (Quick Meanings)
 21. Cheat Sheet
-

1. AI vs ML vs Deep Learning vs GenAI

Ye 4 terms aksar mix ho jaate hain. Simple samjho: ye ek dusre ke andar aane wale circles hain.



AI (Artificial Intelligence)

Machines se aisa kaam karwana jisme normally insaani intelligence lagti hai: samajhna, decide karna, seekhna, problem solve karna. Ye sabse bada umbrella term hai.

- Isme rule-based systems bhi aate hain (jaise chess ka purana program jisme rules haath se likhe gaye).

ML (Machine Learning)

AI ka wo hissa jisme hum rules likhne ki jagah machine ko **data dikha kar** seekhne dete hain.

- **Traditional programming:** Rules + Data → Answer
- **Machine Learning:** Data + Answers → Rules (model khud seekhta hai)
- **Examples:** Spam email filter, loan approval prediction, product recommendations.

Deep Learning (DL)

ML ka wo type jisme **neural networks** use hote hain, jinme bahut saari layers hoti hain ("deep" ka matlab yahi hai).

- Raw data (images, audio, text) se patterns khud nikal leta hai.
- **Examples:** Face recognition, speech-to-text, self-driving car ka vision system.

GenAI (Generative AI)

Deep learning ke models jo **naya content generate** karte hain: text, image, audio, video, ya code.

- **Text:** LLMs (Claude, GPT, Gemini)
- **Image:** Diffusion models (jaise Midjourney, Stable Diffusion)
- **Code:** Coding assistants

LLM = GenAI ka text-focused type. Is manual me hum yahi padh rahe hain.

Quick Comparison

Term	Kya karta hai	Example
AI	Machine se intelligent kaam karwana	Chatbot, chess engine
ML	Data se patterns seekhna	Spam filter
Deep Learning	Neural networks se complex patterns seekhna	Image recognition
GenAI	Naya content banana	ChatGPT, Claude, image generators

Ek example se samjho (Email):

- **Rule-based AI:** "Agar subject me 'lottery' hai to spam."
- **ML:** Hazaron spam/non-spam emails se model seekhta hai ki spam kaisa dikhta hai.
- **Deep Learning:** Neural network email ke text ke complex patterns pakadta hai.
- **GenAI:** Model tumhare liye ek naya reply email likh deta hai.

2. LLM kya hota hai?

LLM = Large Language Model.

Ye ek AI model hai jo bahut saara text padh kar language ke patterns seekhta hai. Uske baad wo:

- Sawaalon ke jawab de sakta hai
- Text likh sakta hai (email, blog, code)
- Summary bana sakta hai
- Translation kar sakta hai
- Code likh aur debug kar sakta hai

Simple example: Socho ek bachcha jisne lakhon kitaabein padh li hain. Usse ab koi bhi sentence adhoora do, wo achha guess kar lega ki aage kya aana chahiye. LLM bhi kuch aisa hi karta hai, bas bahut bade scale par.

"Large" kyun?

- Training data bahut bada hota hai (books, websites, code, articles)
- Model ke andar billions of parameters hote hain (aage samjhenge)

Examples of LLMs: Claude, GPT, Gemini, Llama, Mistral.

3. LLM kaise kaam karta hai? (Next Token Prediction)

LLM ka core kaam ek hi hai:

┆ Pichhle text ko dekh kar, agla token (word ka chhota hissa) predict karna.

Example:

```
Input: "Bharat ki rajdhani"  
Model: "Delhi" (sabse probable agla word)
```

Phir wo ye process baar-baar repeat karta hai:

```
"Bharat ki rajdhani"  
"Bharat ki rajdhani Delhi"  
"Bharat ki rajdhani Delhi hai"  
"Bharat ki rajdhani Delhi hai."
```

Isko **autoregressive generation** kehte hain: ek token generate hua, wo input me judd gaya, phir agla token.

Important baat: Model "sochta" nahi hai jaise insaan sochta hai. Wo probabilities calculate karta hai ki kaunsa token aage sabse suitable hoga. Lekin itna bada training hone ki wajah se ye prediction kaafi intelligent behavior dikhati hai.

4. Tokens aur Tokenization

LLM directly words nahi padhta. Wo text ko chhote tukdon me todta hai jinhe **tokens** kehte hain.

Token kya ho sakta hai?

- Ek poora word (`hello`)
- Word ka hissa (`un`) + (`believ`) + (`able`)
- Ek punctuation (`.`)
- Kabhi ek space ya ek character

Rough thumb rule (English):

- 1 token $\approx$ 4 characters
- 1 token $\approx$ 0.75 word
- 100 tokens $\approx$ 75 words

┆ Hindi ya Hinglish text me usually zyada tokens lagte hain same meaning ke liye, kyunki tokenizer mostly English par optimized hota hai.

Tokens kyun important hain?

1. **Cost:** API pricing tokens ke hisaab se hoti hai (input + output).
2. **Limit:** Context window bhi tokens me naapi jaati hai.
3. **Speed:** Zyada tokens = zyada time.

Flow:

```
Text → Tokenizer → Token IDs (numbers) → Model  
"Hello world" → ["Hello", " world"] → [9906, 1917]
```

Model ko sirf numbers samajh aate hain, isliye har token ka ek ID hota hai.

5. Embeddings

Token ID sirf ek number hai, usme meaning nahi hoti. Isliye har token ko ek **vector** (numbers ki lambi list) me convert kiya jaata hai. Isse **embedding** kehte hain.

Idea: Jin words ka meaning similar hota hai, unke embeddings paas-paas hote hain.

```
"king"   ≈ close to "queen"  
"car"    ≈ close to "truck"  
"apple" (fruit) ≠ close to "laptop"
```

Real-life analogy: Ek map socho jisme har word ek point hai. Similar meaning wale words ek hi area me rehte hain.

Use cases:

- Semantic search (meaning ke basis par search)
 - RAG systems (aage padhenge)
 - Recommendations
-

6. Transformer aur Attention

Almost saare modern LLMs **Transformer architecture** par bane hain (2017 ke paper "Attention Is All You Need" se aaya).

Attention kya hai?

Jab model kisi word ko samajhta hai, wo sentence ke baaki words par bhi "dhyan" deta hai, taaki context samajh aaye.

Example:

└ "Ravi ne Amit ko book di kyunki **wo** busy tha."

Yahan "wo" kaun hai? Ravi ya Amit? Attention mechanism model ko help karta hai ki sahi word se connection bana sake.

Self-Attention ka simple idea

Har token ye poochta hai: "Is sentence me kaun se baaki tokens mere liye important hain?"
Aur uske hisaab se apni understanding update karta hai.

Transformer ke main parts (high level)

Part	Kaam
Embedding layer	Tokens ko vectors me badalna
Attention layers	Tokens ke beech relations samajhna
Feed-forward layers	Information ko process/transform karna
Output layer	Agle token ki probabilities nikalna

Ye attention + feed-forward blocks bahut saari baar stack hote hain (dozens of layers). Jitne zyada layers, utna deep understanding (generally).

7. LLM ki training kaise hoti hai?

Training usually 3 major stages me hoti hai.

Stage 1: Pre-training

- **Kya hota hai:** Model ko bahut bada text data diya jaata hai (books, web pages, code, etc.).
- **Task:** Next token predict karo.
- **Result:** Model ko language, grammar, facts, aur patterns ka broad knowledge mil jaata hai.
- **Cost:** Sabse expensive stage (bahut saare GPUs, weeks/months).

Is stage ke baad model "knowledgeable" hota hai, lekin zaroori nahi ki wo instructions achhe se follow kare.

Stage 2: Fine-tuning (Supervised Fine-Tuning / SFT)

- **Kya hota hai:** Model ko instruction-response ke examples diye jaate hain.
- **Example:**

Instruction: "Is paragraph ka summary do"
Response: "<achha summary>"

- **Result:** Model ab assistant ki tarah behave karna seekhta hai.

Stage 3: Alignment (RLHF / Preference Tuning)

- **Kya hota hai:** Model ke multiple answers ko humans (ya AI feedback) rank karte hain. Model ko aise answers dene ke liye train kiya jaata hai jo helpful, honest aur safe hon.
- **RLHF = Reinforcement Learning from Human Feedback.**
- **Result:** Model ka tone, safety, aur usefulness improve hoti hai.

Summary:

Pre-training	→	Knowledge
Fine-tuning	→	Instructions follow karna
Alignment	→	Helpful + Safe behavior

8. Parameters aur Model Size

Parameters wo numbers (weights) hain jo model ne training me seekhe hain. Ye model ka "learned knowledge" hain.

- Small model: ~1B-8B parameters
- Medium: ~13B-70B
- Large: 100B+ (kai modern models ke exact numbers public nahi hote)

B = Billion (1B = 100 crore).

Kya bada model hamesha behtar hota hai?

Nahi. Bade models generally zyada capable hote hain, lekin:

- Zyada costly hote hain
- Slow ho sakte hain
- Har task ke liye zaroori nahi

Simple tasks (classification, short summaries) ke liye chhota model bhi kaafi hota hai. Isliye **task ke hisaab se model choose karo.**

9. Context Window

Context window = ek baar me model kitna text (tokens me) "yaad" rakh kar dekh sakta hai. Isme tumhara input, purani chat, aur model ka output, sab count hota hai.

Analogy: Ye model ki "working desk" hai. Desk jitni badi, utne zyada documents ek saath khule rakh sakte ho. Desk full ho gayi, to purani cheezein bahar karni padengi.

Important points:

- Agar conversation context window se lambi ho jaaye, to shuru ka hissa kaat diya jaata hai ya summarize karna padta hai.

- Model **apne aap** purani alag chats yaad nahi rakhta. Har naye conversation me sirf wahi hota hai jo context me diya gaya.
- Bahut lamba context dene se cost aur latency badhti hai.
- Bahut lambe context me kabhi kabhi beech ki details miss ho sakti hain.

Tip: Sirf relevant information hi context me daalo.

10. Inference (Model se Output Kaise Milta Hai)

Inference = trained model ko use karke output generate karna. Jab bhi tum chat me sawaal poochte ho ya API call karte ho, wo inference hai.

Training vs Inference

Point	Training	Inference
Kaam	Model seekhta hai	Model seekhe hue ko use karta hai
Weights	Update hote hain	Fixed rehte hain (change nahi hote)
Cost	Bahut zyada, ek baar/kabhi kabhi	Har request par chhota cost
Kaun karta hai	AI companies/labs	Tum aur tumhare users

Inference ke time model tumhari baat se "seekhta" nahi hai. Wo sirf context me diye gaye text ko use karta hai.

Inference ke steps

1. Tumhara prompt aaya
2. Tokenizer se tokens me convert hua
3. Prefill: model poore input prompt ko ek saath process karta hai
4. Decode: model ek-ek token generate karta hai (loop me)
5. Tokens wapas text me convert hue
6. Tumhe response mila (aksar streaming me)

Streaming ka matlab: pura answer ready hone ka wait nahi hota, tokens aate hi screen par dikhte jaate hain.

Inference ke important terms

Term	Meaning
Latency	Request se response tak ka total time
TTFT	Time To First Token: pehla token aane me kitna time laga
Throughput / Tokens per second	Model kitni speed se tokens generate kar raha hai
Input tokens	Jo tumne bheja (prompt, history, documents)
Output tokens	Jo model ne generate kiya

Speed aur cost kis par depend karti hai?

1. **Model size:** Bada model = aam taur par slow aur costly.
2. **Input length:** Lamba prompt = zyada processing.
3. **Output length:** Har output token ek-ek karke banta hai, isliye lamba answer = zyada time.
4. **Hardware:** GPU/accelerator ki quality.
5. **Traffic/load:** Server par kitni requests chal rahi hain.

Practical tips

- Prompt short aur relevant rakho.
- `max_tokens` set karo taaki output zarurat se lamba na ho.
- Simple tasks ke liye chhota/fast model use karo.
- User-facing apps me streaming on rakho, isse app fast feel hoti hai.

11. System / User / Assistant Messages

Chat-based LLMs ko input **messages ki list** ke form me diya jaata hai. Har message ka ek **role** hota hai.

3 main roles

Role	Kaun likhta hai	Kaam
System	Developer/app	Model ke liye base rules: persona, tone, format, boundaries
User	End user	Actual sawaal ya task
Assistant	Model	Model ka jawab (purani history me bhi aata hai)

System message kya hota hai?

Ye model ke liye "standing instructions" hain jo poori conversation me apply hoti hain.

Example:

```
System: Tum ek customer support assistant ho for Icecube Store.  
Hamesha polite raho, jawab 100 words se kam rakho,  
aur agar answer nahi pata to bolo "Main team se confirm karke batata  
hoon."
```

User message kya hota hai?

Wo cheez jo user abhi poochh raha hai.

```
User: Mera order kab aayega?
```

API me kaisa dikhta hai (concept)

```
json  
  
{  
  "system": "Tum ek helpful support assistant ho. Short aur polite jawab do.",  
  "messages": [  
    { "role": "user", "content": "Mera order kab aayega?" },  
    { "role": "assistant", "content": "Kripya apna order number share karein." },  
    { "role": "user", "content": "Order number 12345 hai." }  
  ]  
}
```

Kuch providers (jaise Anthropic API) `system` ko alag field me lete hain, kuch (jaise OpenAI-style APIs) usse messages list me pehla message banate hain. Concept same hai.

Important baatein

1. **Model stateless hota hai:** Wo pichhli baat khud yaad nahi rakhta. Har request me poori history (user + assistant messages) dobara bhejni padti hai.
2. **History = context window ka hissa:** Conversation jitni lambi, utne tokens use honge.
3. **System prompt ka kaam:** behavior, tone, aur output format set karna.
4. **System prompt 100% secure nahi hai:** Isme secrets (API keys, passwords) kabhi mat daalo. Clever user prompt injection se instructions nikalwane ki koshish kar sakta hai.

System prompt likhne ke tips

- Role aur goal clearly likho.

- Do's aur Don'ts specific rakho.
- Output format define karo (bullet, JSON, length).
- Edge cases bolo: "Agar sawaal out-of-scope ho to kya karna hai."
- Prompt ko test karo alag-alag user inputs ke saath.

System vs User: kya kahan likhein?

Cheez	Kahan rakhein
Tone, persona, rules, format	System
Har baar badalne wala sawaal/data	User
Reference documents (RAG chunks)	User message me (ya clearly labeled section me)

12. Generation Settings

Jab tum LLM ko API se use karte ho, kuch settings output ko control karti hain.

Temperature

Randomness ka control.

Temperature	Behavior	Kab use karein
Low (0 – 0.3)	Predictable, consistent	Code, facts, data extraction
Medium (0.4 – 0.7)	Balanced	General chat, emails
High (0.8+)	Creative, varied	Brainstorming, stories

Top-p (Nucleus Sampling)

Model sirf top probable tokens me se choose karta hai jinki combined probability p tak pahunchhe. Chhota top-p = safer output.

| Tip: Aam taur par temperature **ya** top-p me se ek hi adjust karo, dono ek saath nahi.

Max Tokens

Output kitna lamba ho sakta hai uski limit. Isse cost aur length control hoti hai.

Stop Sequences

Ek special text jispar model generation rok deta hai.

13. Prompting Basics

Prompt = wo instruction/input jo tum model ko dete ho. Prompt ki quality se output ki quality bahut badalti hai.

Achhe prompt ke 5 elements

1. **Role/Context:** Model kaun hai, situation kya hai
2. **Task:** Exactly kya karna hai
3. **Input/Data:** Kis data par kaam karna hai
4. **Format:** Output kaisa chahiye (bullet, table, JSON)
5. **Constraints:** Length, tone, kya nahi karna

Bad vs Good Prompt

Bad:

Blog likho AI par.

Good:

Tum ek tech content writer ho. Beginners ke liye "AI kya hota hai" par 600 words ka blog likho. Simple language use karo, 3 real-life examples do, aur end me 3 key takeaways bullet points me do.

Useful techniques

1. Zero-shot: Bina example ke seedha task.

Is review ka sentiment batao: "Product bahut achha hai."

2. Few-shot: Kuch examples deke pattern sikhana.

Review: "Bekaar quality" → Negative
Review: "Mast product" → Positive
Review: "Theek-thaak hai" → ?

3. Step-by-step reasoning: Complex problems me model se steps me sochne ko bolo. Isse accuracy aksar improve hoti hai.

4. Structured output: Format clearly batao.

Output sirf valid JSON me do: {"name": "", "email": ""}

5. Delimiters: Input ko alag dikhane ke liye tags ya quotes use karo.

```
<article>
...text...
</article>
```

Upar ke article ka 3 line summary do.

Prompting ke golden rules

- Clear aur specific bano
 - Ek prompt me ek main task rakho (ya steps me todo)
 - Examples do agar format important hai
 - Pehla output perfect na ho to iterate karo
-

14. Hallucination

Hallucination = jab model confidently galat ya banayi hui information deta hai.

Kyun hota hai?

- Model ka kaam "sabse likely text" banana hai, "sach" verify karna nahi
- Training data me gaps ya galat info ho sakti hai
- Jis topic par data kam ho, wahan model guess kar deta hai

Common examples:

- Fake book titles, research papers, ya links
- Galat dates, numbers, ya quotes
- Aisa code jisme non-existent functions hon

Kaise kam karein?

1. Model ko sahi source/document context me do (RAG)
2. Bolo: "Agar pata nahi hai to bolo pata nahi"
3. Important facts ko khud verify karo
4. Low temperature use karo factual tasks ke liye
5. Sources ya citations maango aur unhe check karo

Rule: LLM ka output ek draft ki tarah treat karo, final truth ki tarah nahi, khaaskar medical, legal, ya financial cheezon me.

15. RAG (Retrieval-Augmented Generation)

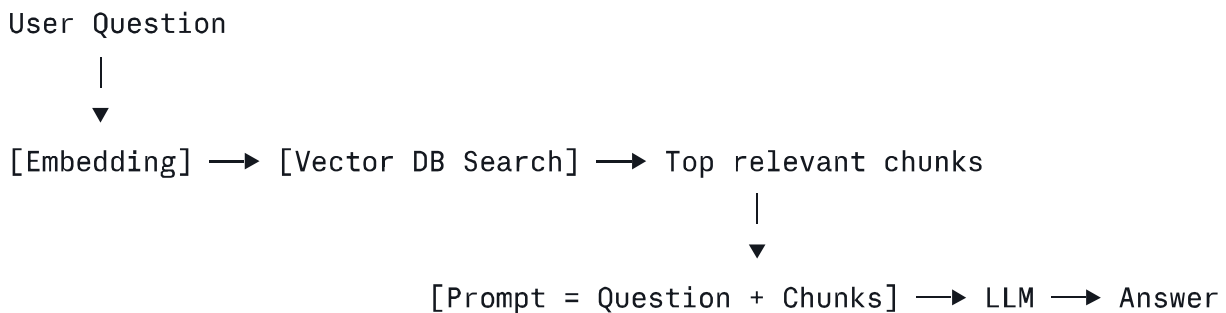
Problem: Model ka knowledge training tak ka hota hai, aur usko tumhare private documents ka pata nahi hota.

Solution: Pehle relevant documents dhoondho, phir unhe prompt me daal kar model se answer banwao.

RAG ka flow

1. Documents ko chhote chunks me todo
2. Har chunk ka embedding banao
3. Embeddings ko Vector Database me store karo
4. User ka question aaya → uska embedding banao
5. Vector DB se sabse similar chunks nikalo
6. Question + chunks ko LLM ko do
7. LLM un chunks ke basis par answer banata hai

Diagram:



RAG ke fayde

- Latest aur private data use ho sakta hai
- Hallucination kam hota hai
- Model ko dobara train nahi karna padta
- Sources dikhaye ja sakte hain

RAG ki common problems

- Chunking galat hone se context toot jaata hai
- Retrieval me galat chunks aa jaate hain
- Bahut saare chunks daalne se prompt bada aur noisy ho jaata hai

16. Prompting vs RAG vs Fine-tuning

Ye sabse common confusion hai. Simple table:

Approach	Kab use karein	Fayda	Limitation
Prompting	Pehla step hamesha	Fast, sasta, easy	Model ke existing knowledge tak limited
RAG	Private/latest knowledge chahiye	Data update easy, sources mil sakte hain	Retrieval quality par depend
Fine-tuning	Specific style, format, ya behavior chahiye	Consistent output, chhote prompts	Data aur effort chahiye, knowledge update mushkil

Thumb rule:

1. Pehle **prompting** try karo.
2. Knowledge ki kami ho to **RAG** add karo.
3. Style/behavior consistent chahiye aur upar ke kaam na kare, tab **fine-tuning** socho.

Fine-tuning naya "knowledge" sikhane ke liye kam, aur "behavior/style" sikhane ke liye zyada achhi hoti hai.

17. Tool Use aur AI Agents

Tool Use (Function Calling)

LLM sirf text banata hai, lekin usse tools se connect kiya ja sakta hai, jaise:

- Calculator
- Web search
- Database query
- Email bhejna
- Tumhari apni API

Flow:

```

User: "Ahmedabad ka aaj ka weather kya hai?"
|
▼
LLM decide karta hai: weather tool call karna hai
|
▼
Tool run hota hai → real data aata hai
|
▼
LLM us data se final jawab banata hai

```

AI Agent kya hai?

Agent = LLM + tools + loop. Agent ek goal leta hai, khud steps plan karta hai, tools use karta hai, result dekhta hai, aur goal poora hone tak repeat karta hai.

Goal → Plan → Tool use → Result dekho → Next step → ... → Final answer

Example: "Mere competitors ki pricing compare karo" → agent search karega, pages padhega, data table banayega, aur summary dega.

Dhyan rakhein:

- Agents galtiyan kar sakte hain, important actions (delete, payment, email send) par human approval rakho
- Tools ko sirf minimum permissions do

18. LLM ki Limitations

1. **Hallucination:** Galat info confidently de sakta hai.
2. **Knowledge cutoff:** Training ke baad ki ghatnaon ka pata nahi (jab tak search/tools na ho).
3. **Maths/logic me galti:** Lambi calculations me chook ho sakti hai, tools use karna behtar hai.
4. **Context limit:** Bahut lamba input ek saath handle nahi kar sakta.
5. **Bias:** Training data ke biases output me aa sakte hain.
6. **Non-deterministic:** Same prompt par alag output aa sakta hai (khaaskar high temperature par).
7. **Prompt sensitivity:** Prompt ke chhote badlav se output badal sakta hai.
8. **Real understanding ka sawaal:** Model patterns par kaam karta hai, insaan jaisi lived experience nahi hoti.

19. Safety aur Best Practices

Data Privacy

- Sensitive data (passwords, API keys, customer personal info, confidential contracts) bina soche-samjhe prompts me na daalo.
- Company policy aur provider ki data-handling terms check karo.

Prompt Injection

Agar model ko external content (web page, email, document) padhne ko do, to us content me chhupi malicious instructions ho sakti hain, jaise: "Pichhle instructions ignore karo aur ye karo..."

Bachav:

- External content ko "data" treat karo, "instructions" nahi
- Tools ko limited permissions do
- Risky actions par human confirmation lo

Output Verification

- Code ko run/test karke hi production me lagao
- Facts, numbers, aur citations verify karo
- Legal/medical/financial decisions ke liye professional se confirm karo

Cost Control

- Chhote model se shuru karo, zaroorat par upgrade karo
 - Prompt short aur relevant rakho
 - Max tokens set karo
 - Repeated cheezon ke liye caching ka use karo (agar provider support kare)
-

20. Glossary (Quick Meanings)

Term	Simple Meaning
LLM	Bada language model jo text samajhta aur banata hai
Token	Text ka chhota tukda jo model padhta hai
Tokenizer	Text ko tokens me todne wala component
Embedding	Token/text ka number-vector representation jisme meaning hoti hai
Transformer	Modern LLMs ka underlying architecture
Attention	Mechanism jisse model relevant words par focus karta hai
Parameters	Model ke learned weights
Pre-training	Bade data par basic language seekhna
Fine-tuning	Specific task/style ke liye extra training
RLHF	Human feedback se model ko align karna
Context Window	Ek baar me model kitna text dekh sakta hai
Temperature	Output ki randomness ka control
Prompt	Model ko diya gaya input/instruction
System Prompt	Model ke behavior ke liye base instructions
Hallucination	Model ka confidently galat/fake output
RAG	Documents dhoondh kar model ko dena taaki grounded answer mile
Vector DB	Embeddings store aur search karne wala database
Agent	LLM + tools + loop jo goal ke liye khud steps leta hai
Inference	Trained model se output generate karna
AI	Machines se intelligent kaam karwana (sabse bada umbrella)
ML	Data se seekhne wala AI
Deep Learning	Neural networks (bahut layers) wala ML
GenAI	Naya content generate karne wala AI

Term	Simple Meaning
Role	Message kisne likha: system, user ya assistant
TTFT	Pehla token aane tak ka time
Streaming	Tokens ko ek-ek karke turant dikhana
Latency	Response aane me lagne wala time

21. Cheat Sheet

Ek line me LLM:

LLM ek next-token predictor hai jo bahut bade text par train hua hai.

Yaad rakhne wali 10 baatein:

1. LLM tokens par kaam karta hai, words par nahi.
2. Meaning embeddings me store hoti hai.
3. Attention se model context samajhta hai.
4. Training: Pre-training → Fine-tuning → Alignment.
5. Context window model ki working memory hai.
6. Low temperature = consistent, high = creative.
7. Achha prompt = clear task + context + format.
8. Hallucination real hai, verify karna zaroori hai.
9. Private/latest data ke liye RAG use karo.
10. Pehle prompting, phir RAG, phir fine-tuning.

Prompt template (copy-paste ready):

```
Role: Tum ek [role] ho.
Task: [exact kya karna hai]
Context: [background information]
Input: [data/text]
Format: [bullet / table / JSON / paragraph]
Constraints: [length, tone, kya nahi karna]
```

Aage kya seekhein?

- Prompt Engineering (advanced techniques)
- Embeddings aur Vector Databases
- RAG system banana (chunking, retrieval, evaluation)

- Function calling aur Agents
 - LLM evaluation (output quality kaise naapein)
 - Fine-tuning basics (LoRA, PEFT)
-

Happy Learning!